Examiner Q&A / Presenter Guide

Self-Driving Mini Car • Krishna Menon, Aditya Rana, Parth Surve
Companion to the 16-slide final presentation • 30 October 2026

Use these answers to explain design choices and distinguish implemented software, logged evaluation and physical validation still required. This is preparation material; it is not a claim to predict every examiner question.

Rubric focus	Where the deck answers it
A — Literature / problem	Slides3–4: objectives, prior work and integration gap
B — System design	Slides5–9: block, flow, use cases, interfaces and lane geometry
C — Implementation / results	Slides8–12: obstacle/lane/pothole/sign algorithms and scores
D — Technical depth / future	Slide13 and speaker notes: limitations and next experiments
E — Presentation delivery	16slides; structured visuals; approximately13.5minutes
F — Viva understanding	This guide; code/log provenance in notes and slide12 links

Suggested delivery: 1–2:35s; 3:50s; 4:60s; 5:75s; 6:65s; 7:40s; 8:55s; 9:80s; 10:75s; 11:85s; 12:80s; 13:45s; 14:35s; 15:20s; 16:10s. Total13min30s.

Architecture and project choices

01. What exactly is the contribution?

A modular, controlled-track RC-car system that coordinates lane geometry, learned hazards, sonar and embedded motion execution. The contribution is the integration and explicit command/safety boundary, together with reproducible training artifacts. It is not a new YOLO architecture or demonstrated public-road autonomous driving.

02. Why Raspberry Pi plus Arduino instead of one board?

The Pi supports camera capture, Python/OpenCV and neural inference. Uno executes motor PWM, command parsing, sonar timing and local finite-state behavior. This separates compute-heavy interpretation from embedded actuation. The split reduces coupling; it does not prove deterministic end-to-end latency without measurements.

03. Which camera is actually used, and how is orientation handled?

The final report records an OV5647 replacement after the originally planned global-shutter camera failed. The runtime requests 640 × 480 and attempts Picamera2 hflip=1, vflip=1, equivalent to 180° rotation. This compensates the assumed mounting orientation. Verify the live image is upright and left/right labels correspond physically before track testing; fallback capture configurations do not automatically prove the same orientation.

04. How does information reach the motors?

CSI frame → Pi perception → action arbitration → line-oriented ASCII command over serial at 115200 baud → Uno command parser and motion state machine → L298N PWM/direction outputs → paired left/right motors. Uno telemetry returns distances/state to Pi. The dashboard exposes high-level state and controls.

05. Is this four independent motor control or car-like steering?

Each L298N channel drives a side pair of TT motors. Turning is skid-steer behavior, not front-wheel servo steering. The software angle is a motion command or time-calibration input; it is not a measured steering-wheel angle. Wheel slip and surface friction affect actual rotation.

06. What is implemented, and what is still future work?

Source implements the two-layer runtime, lane logic, pothole/obstacle inference, follow behavior and dashboard. The four-sign extension has trained PT/NCNN artifacts and offline image-to-command checks, supplied separately from Krishna main 9aeb238. Physical timing, sign stopping behavior, lane accuracy and turn-angle accuracy remain unverified in the supplied evidence. Encoders and completed IMU heading feedback are future work.

Algorithms and perception

07. How does obstacle avoidance choose between slowing, stopping and bypassing?

YOLO boxes and pothole/person detections are checked for a forward-corridor hazard. In current main.py, a central box taller than40% of the frame requests STOP; above15% requests caution. Zone scoring then combines confidence, obstacle-type weight, box extent, bottom-edge position and front sonar into left/center/right danger. A clearer side can receive PI_AVOID and a timed skid-steer turn; all-blocked or imminent conditions suppress normal guidance. Uno reflex states take control when active. Release/escape logic is mode-specific, including guarded autonomous unstick behavior; this is not a claim of a formally verified safety controller.

08. Why use classical vision for lanes?

A controlled chart-paper track provides useful color/contrast structure. ROI masking, window search and polynomial fitting avoid another neural inference workload. This is a pragmatic track-specific choice, not proof that classical vision generalizes better than learned lane detection.

09. Explain the lane pipeline and steering equation.

The lower 55% ROI is color-masked; hazard boxes are excluded. Sliding windows collect line pixels and robust quadratic fitting estimates $x(y)=ay^2+by+c$ in normalized image coordinates. At $y=0.68$, $e=x_{target}-0.5$ and tangent heading $\psi=-\operatorname{atan}(2ay+b)$, in degrees. Command $\theta=\operatorname{clip}[145e+(28/45)\psi, -38°, +38°]$. This is the implemented image-geometry rule, not a calibrated metric bicycle-model controller.

10. What happens if a lane line disappears?

The implementation supports center estimation from a single line, temporal smoothing and limited memory. EMA alpha is 0.45; large lateral jumps are rejected. When missing beyond the configured 8-frame limit, the runtime stops. Eight frames is a frame count, not a fixed millisecond latency; the duration depends on actual processing rate.

11. How are potholes detected if ultrasound cannot see a flat print?

The dedicated visual pothole detector identifies image regions; lower-image and bounding-box filtering feed hazard scoring. Sonar provides complementary nearby-obstacle sensing. A flat printed pothole is a visual test artifact, not a physical depression; sonar is not evidence of depression detection. Classical Laplacian CV is available but off by default.

12. Why do road and printout pothole scores differ so much?

They are distinct validation domains and runs. Final RDD mAP50 is 0.25226, while the printout run logs 0.97333. Printouts can be visually narrower and easier; dataset composition, split independence and domain shift must be examined before explaining the entire difference. Do not treat the two numbers as a controlled before/after improvement on the same test set.

13. How does follow-me fit into the architecture?

The runtime includes a KCF-based person/owner tracking subsystem with target selection, appearance checks and reacquisition logic. The same higher-level safety and motor interfaces are used. The deck focuses on lane, traffic and potholes as requested; it does not claim a new follow-me benchmark or identity-recognition accuracy.

Traffic signs and real-car behavior

14. Is traffic-sign recognition trained, or is it color matching?

It is a four-class YOLOv10n fine-tune with actual weights and an FP16 NCNN export. Classes are STOP, No Entry, Left and Right. Training used generated sign imagery and varied backgrounds. A dedicated duplicate-box filter handles overlapping/conflicting outputs before policy logic; it is not simply matching a red shape to a motor command.

15. What data and training settings were used?

768 training images, 128 validation images and 128 test images; 25 epochs, input size 320, batch 8, AdamW, frozen early layers, and no horizontal/vertical flips that would alter arrow meaning. The images are synthetic. Counts are images, not independent physical camera trials. Check the model card and generator for reproducibility.

16. Will a printed STOP sign definitely stop the moving car?

The code is intended to recognize camera frames of the sign, confirm three fresh observations and command a stopped dwell. Synthetic examples and offline policy checks support that software path. They do not establish a guarantee under the real camera, lighting, viewing angle, speed and braking dynamics. A controlled, low-speed physical test with a separate operator stop is still required to establish the observed outcome.

17. What does each detected sign do?

STOP requests a 3-second stopped dwell; the dwell accrues only while the runtime commands stopped and not during Uno avoidance. No Entry holds until explicit reset. Left/Right create a pending turn, acted on only after geometry and safety clearance. A turn is not automatically authorized just because a card exists somewhere in the image.

18. Why three confirmations and freshness limits?

One prediction can flicker or be false. The policy needs three consecutive fresh observations, not three reused cached boxes. Default sign polling is 4 Hz with a maximum configurable 5 Hz; the actual achieved rate depends on compute contention. A gap beyond 0.75 s clears confirmation history. At ideal 4 Hz, three observations span roughly 0.5 s from first to third, plus capture/inference/control delays.

19. How do camera angle and apparent size affect a turn?

Default horizontal FOV is 53.5°. The pinhole model uses fx=W/[2tan(HFOV/2)] and bearing=atan[(box_center_x−W/2)/fx]. A turn gate requires box height/H ≥0.12 and absolute bearing ≤22°. These are tunable apparent-proximity and direction heuristics. They do not measure distance or locate the junction; camera pitch alone cannot project an elevated sign onto the road plane.

20. Does a left command prove an accurate 90-degree physical turn?

No. Autonomous/follow modes use timed ±90° commands with PWM210 and a default 10ms/degree calibration. Lane mode uses 12° pulses, requires at least72 commanded degrees plus observed lane departure and three fresh centered DUAL_LOCKED frames, and faults at96 commanded degrees or6s without completion. Actual yaw needs measured feedback; time calibration depends on battery, surface and slip.

21. What happens with hazards, stale sensors or conflicting signs?

The extension yields to existing obstacle decisions when idle, suppresses ordinary guidance during sign-owned holds/maneuvers, and faults if an active turn is interrupted. Clearance requires fresh obstacle results (≤0.5s) and sonar (≤0.8s). Missing sign sensing inhibits sign-driven movement. Simultaneous left/right observations do not authorize a guessed turn; near-confidence conflicts remain ambiguous.

22. What failure cases were actually recorded?

One held-out negative red SLOW octagon produced a false STOP, and a small unfamiliar narrow-font STOP was missed in the challenge set. PT and NCNN passed four class-to-action checks using mocked motors. These known cases motivate real-image collection and targeted negative examples; a high synthetic mAP does not erase them.

Results, validation and hardware

23. What do precision, recall, IoU and mAP mean?

Precision=TP/(TP+FP); recall=TP/(TP+FN). IoU is bounding-box intersection area divided by union area. AP summarizes a precision–recall curve for a class. mAP averages AP across classes. mAP50 evaluates at IoU0.50; mAP50–95 averages IoU thresholds0.50–0.95. A detector confidence value is not the probability that the complete car maneuver succeeds.

24. Which exact scores are defensible?

Final logged RDD: precision0.53992, recall0.27092, mAP50=0.25226, mAP50–95=0.11421. Printout:0.99902,0.95506,0.97333,0.70171 respectively. Synthetic signs:0.98462,0.99896,0.99486,0.97783. The deck links row labels to source logs. RDD contains50logged epochs; printout contains6recorded rows although args request25.

25. Why is there no lane mAP score?

The lane module produces geometry rather than learned object boxes with class labels. Better metrics are centerline lateral error, track completion, loss/recovery rate, intervention count and processing latency. These need annotated frames or physical reference measurements; no measured lane-accuracy percentage is invented here.

26. Does it run 20–30 fresh detections per second?

The supplied sources do not provide dated raw Pi logs verifying that claim. The report includes timing figures, but loop FPS, model-only FPS and fresh inference FPS differ. General neural detection is scheduled every third loop, and the sign detector is separately rate-limited. With multiple models, actual throughput must be measured on Pi under the complete runtime workload.

27. What latency and stopping-distance experiment should be run?

Log frame-capture time, preprocessing, inference completion, decision time, serial write and command state. Observe physical brake onset and distance independently. Repeat at several PWM settings, lighting conditions and battery states. Report medians and upper-tail latency, misses/false stops, and distance distributions. Do not sum unmeasured component guesses and call the result a benchmark.

28. What are the exact Uno pins and current timing defaults?

Left motor pair/channelB: D6PWM, D10/D11 direction. Right pair/channelA: D5PWM, D8/D9. Left sonar D7TRIG/D13ECHO; right D12TRIG/D4ECHO. Current sketch defaults include a5000ms watchdog and300ms sonar-pair interval; EEPROM settings can override. Some paper sections state different values, so effective firmware and tuning are the implementation reference.

29. Why two supplies, and are the boards isolated?

The documented design separates Pi power from motor-side supply to reduce load coupling. The motor/Arduino rail uses the documented battery/buck arrangement; Pi uses a dedicated power bank. Ordinary connected USB does not provide galvanic isolation. A stable supply and required common references do not eliminate noise, grounding or brownout concerns; those are physical integration matters.

30. What is the correct paper status and SDG claim?

The team confirms submitted; venue, identifier and decision notification were not supplied. Do not claim accepted or under review without evidence. SDG9.5is the strongest research/innovation link. SDG3.6and11.2are supporting safety/mobility research motivations. No measured accident reduction, carbon saving or transport deployment is claimed.

Source trail

Krishna repository: github.com/menonkrishna57/model-car-capstone/tree/9aeb238. Effective runtime: car/main.py, car/perception/config.py, lane_detector.py, detector.py and Arduino sketch-local pins.h. Pothole metrics: pothole_detection/runs_rdd/{custom_pothole_model,printout_gpu_pothole_model}/results.csv. Trained sign extension: MODEL_CARD.md; validation/heldout_summary.json; validation/action_check_ncnn.json; car/perception/traffic_signs.py; car/control/traffic_sign_policy.py; car/main.py. Extension download branch: github.com/parthsurve2004-svg/model-car-capstone/tree/downloads/traffic-sign-pi. Supporting documents: user-supplied final paper and68-page report.